

Aula 04 - Fundamentos da Linguagem C Embarcada

Descrição: Estudo de sintaxe básica, tipos de dados, variáveis, operadores matemáticos/lógicos e boas práticas de código limpo em C.

Objetivo: Compreender a sintaxe essencial da linguagem C aplicada a microcontroladores, dominando o uso de tipos de dados, variáveis, operadores e as premissas de código limpo para sistemas com recursos restritos.

Bloco 1 — Introdução ao C para Sistemas Embarcados

1.1 O Papel do Compilador e o Fluxo de Execução

Na programação de microcontroladores (como a plataforma Arduino), o código escrito no computador é chamado de **sketch**. Esse código é redigido em um dialeto baseado na linguagem C/C++ (frequentemente associado à linguagem de programação *Wiring*). No entanto, microcontroladores não conseguem ler texto diretamente; eles compreendem apenas níveis lógicos de tensão e instruções em formato binário.

O **compilador** é o utilitário de software responsável por traduzir o código-fonte legível por humanos para o **código de máquina** (arquivos binários contendo sequências de 0s e 1s).

- Quando clicamos no botão **Verify (Verificar)** na IDE do Arduino, o compilador apenas analisa a sintaxe do programa para garantir que ele respeita rigorosamente as regras estruturais da linguagem C.
 - Ao clicar no botão **Upload (Carregar)**, o processo de compilação é concluído, o arquivo binário gerado é transferido através do cabo USB e gravado permanentemente na memória do chip de controle do hardware.
-

1.2 A Estrutura Básica de um Sketch: as Funções `setup()` e `loop()`

Um programa para microcontrolador obrigatoriamente deve conter duas funções fundamentais pré-definidas para que possa compilar com sucesso:

```
void setup() {  
    // Código executado apenas uma única vez  
}  
  
void loop() {  
    // Código executado repetidamente em ciclo infinito  
}
```

- **A Função `setup()`:** Executada apenas **uma única vez** no momento em que a placa é ligada, quando o botão *Reset* é pressionado ou após a transferência de um novo código. Destina-se às **configurações iniciais do sistema**, como a definição dos pinos do microcontrolador que atuarão como entrada ou saída (`pinMode`), inicialização de taxas de transmissão de barramentos seriais (`Serial.begin`) e acionamento inicial de periféricos.
- **A Função `loop()`:** Entra em execução sequencial imediatamente após o término de `setup()`. Seu comportamento é o de um **laço de repetição infinito (ciclo de amostragem)**. Ela executa as linhas de

comando de cima para baixo, passo a passo; ao atingir a chave de fechamento `}`, o programa reinicia a leitura a partir da primeira instrução do bloco. É onde ocorrem as leituras contínuas de sensores, o processamento de lógica de controle e o acionamento dinâmico de atuadores.

1.3 Gerenciamento de Memória e Restrições de Hardware

Diferente de um computador pessoal, que dispõe de gigabytes de RAM e armazenamento, os microcontroladores operam sob severas limitações físicas de recursos computacionais:

1. **Memória Flash (Não Volátil):** Destina-se ao armazenamento do código binário do programa (o *sketch* compilado). Os dados aqui guardados não são perdidos quando o dispositivo é desligado.
 - No microcontrolador **ATmega328** (utilizado no Arduino Uno), há apenas **32 kB de Flash** (sendo que 2 kB são reservados para o *bootloader* de inicialização do chip, restando 30 kB úteis).
 - No **ATmega2560** (Arduino Mega), dispõe-se de **128 kB de Flash** (com 4 kB dedicados ao *bootloader*).
 2. **Memória SRAM (Estática de Acesso Aleatório — Volátil):** Utilizada como memória de trabalho para armazenar variáveis temporárias de processamento, pilhas de funções e dados de comunicação. É extremamente escassa:
 - O **ATmega328 possui somente 2 kB (2.000 bytes) de SRAM**.
 - **Consequência Crítica:** Se o programador utilizar tipos de dados muito grandes ou arrays gigantescos, a memória SRAM se esgotará rapidamente. **O Arduino não emite alertas de falta de memória durante a execução do programa; ele simplesmente trava ou reinicia sem aviso prévio.**
 - *Soluções de otimização:* Para economizar SRAM, podemos forçar o armazenamento de dados imutáveis (como fontes de caracteres ou mensagens longas de texto) diretamente na memória Flash utilizando o modificador especial **PROGMEM** (necessitando importar a biblioteca `<avr/pgmspace.h>`).
 3. **Memória EEPROM (Não Volátil e Gravável):** Utilizada para persistência de dados de calibração que devem sobreviver à falta de energia elétrica (como valores máximos ou configurações do usuário), mas que precisam ser alterados eventualmente por software.
-

1.4 Boas Práticas Iniciais: Código Limpo Embarcado

Desenvolver código limpo (*clean code*) em hardware de recursos restritos é essencial para a manutenção e estabilidade do projeto:

- **Nomenclatura de Variáveis (Convenção CamelCase):** Nomes de variáveis devem ser descritivos (mostrando claramente sua utilidade no circuito) e não conter espaços em branco ou caracteres especiais. A convenção amplamente recomendada dita que variáveis e funções iniciem com **letras minúsculas** e, caso o nome seja composto, as palavras seguintes comecem com **letras maiúsculas** (exemplo: `ledPin` ou `tempoEsperaAlerta`).
- **Indentação e Espaços em Branco:** O compilador ignora completamente recuos de tabulação (indentações), quebras de linha ou espaços redundantes. Eles servem exclusivamente para garantir a leitura humana. Sempre organize o código com chaves alinhadas para delimitar os escopos de forma legível.
- **Comentários Descritivos:** Servem para explicar o funcionamento e a intenção da lógica, sendo ignorados pelo compilador. Em C, utilizamos:
 - `//` para **comentários de linha única**.
 - `/*` e `*/` para **comentários multilinha ou blocos de código**.

Bloco 2 — Tipos de Dados e Variáveis

2.1 Tipos Primitivos na Arquitetura AVR (8 bits)

Uma variável é uma região nomeada da memória SRAM onde podemos armazenar informações durante o processamento do programa. Em C, é obrigatório declarar o **tipo exato de dado** que a variável armazenará. A tabela abaixo exibe os tipos primitivos na arquitetura de microcontroladores AVR (como o Arduino Uno):

Tipo de Dado	Tamanho na SRAM	Faixa de Valores	Aplicação Recomendada
boolean / bool	1 byte	true (1) ou false (0)	Flags de estado (ex: <code>ledLigado = true</code>).
char	1 byte	-128 a 127	Armazenar caracteres individuais ASCII (entre aspas simples ' ').
byte	1 byte	0 a 255	Controle de pinos físicos e comunicação serial de bytes.
int	2 bytes (16 bits)	-32.768 a 32.767	Leituras de ADC (0 a 1023), cálculos e contadores pequenos.
unsigned int	2 bytes (16 bits)	0 a 65.535	Contadores estritamente positivos de até 16 bits.
long	4 bytes (32 bits)	-2.147.483.648 a 2.147.483.647	Cálculos matemáticos grandes.
unsigned long	4 bytes (32 bits)	0 a 4.294.967.295	Medições de tempo longas via função <code>millis()</code> .
float	4 bytes (32 bits)	$\pm 3.4028235 \times 10^{38}$	Números reais/ponto flutuante (cálculos de engenharia e física).
double	4 bytes (32 bits)	O mesmo que o <code>float</code>	No Arduino Uno, o <code>double</code> não possui precisão dupla de 8 bytes (é processado exatamente como <code>float</code>).

2.2 Modificadores de Tipo

Os modificadores alteram a forma como o compilador reserva espaço e gerencia o bit de sinal de uma variável:

- **signed:** Modo padrão. Indica que a variável pode receber tanto valores positivos quanto negativos, mantendo o bit mais significativo dedicado à representação do sinal algébrico.
- **unsigned:** Elimina o bit de sinal de variáveis do tipo `char` ou `int`, permitindo armazenar apenas números positivos e o zero. **Como consequência, dobra-se a capacidade positiva de representação de valor** (ex: o `char` vai de -128 a 127, enquanto o `unsigned char` vai de 0 a 255).
- **short:** Garante que uma variável inteira terá tamanho fixo de 16 bits.
- **long:** Expande o tamanho de variáveis inteiras para 32 bits (ou indica maior precisão para floats em sistemas não-AVR).

Regra de Ouro para Recursos Restritos:

Escolher sempre o menor tipo de dado suficiente para a tarefa é um princípio de otimização crucial.

Exemplo Prático: Armazenar o número de uma porta física do Arduino (que varia de 0 a 13 no Uno ou de 0 a 54 no Mega) em uma variável do tipo `int` consome **2 bytes** de SRAM. Ao declarar essa variável como tipo `byte`, consome-se apenas **1 byte**, poupando metade da memória alocada para aquele pino.

2.3 Escopo de Variáveis: Locais versus Globais

O local de declaração de uma variável define a visibilidade (escopo) e o seu tempo de existência na memória do dispositivo:

```
int pinoSensor = A0; // VARIÁVEL GLOBAL: Visível em todo o código.

void loop() {
    int leituraLocal = analogRead(pinoSensor); // VARIÁVEL LOCAL: Existe apenas no ciclo do loop().

    static int contadorBipes = 0; // LOCAL ESTÁTICA: Visível apenas no loop(), mas não perde seu valor.
    contadorBipes++;
}
```

1. **Variáveis Globais:** Declaradas fora de qualquer função (no topo do sketch). Elas permanecem na memória SRAM durante todo o tempo de funcionamento da placa e podem ser acessadas e modificadas por qualquer função do programa.
 - **Perigo elétrico/lógico:** O uso massivo de variáveis globais aumenta desnecessariamente o consumo contínuo da escassa RAM. Além disso, dificulta o diagnóstico de erros, já que qualquer trecho isolado do programa pode alterar o valor da variável de forma inesperada.
 2. **Variáveis Locais:** Declaradas dentro de um bloco delimitador (como o interior de uma função ou de um laço `for`). São criadas no momento em que a função entra em execução e destruídas imediatamente quando ela se encerra, liberando espaço de memória. Elas não podem ser vistas ou alteradas por outras funções externas.
 3. **Variáveis Locais Estáticas (`static`):** Se declararmos uma variável dentro de uma função precedida pelo qualificador `static`, ela se comportará como uma variável local (visível apenas dentro daquela função), mas **não será destruída ao fim da execução**. Ela reterá seu valor acumulado de modo que, na próxima vez que a função for chamada, o dado estará preservado, sem a necessidade de criar uma variável global.
-

2.4 Diretiva `#define` versus Qualificador `const`

Para associar nomes legíveis a pinos ou valores imutáveis de calibração que não mudam durante a execução, existem dois caminhos em C:

- **A Diretiva `#define`:** É uma instrução direcionada ao pré-processador (executada momentos antes da compilação real). Ela simplesmente localiza todas as ocorrências textuais do identificador no código e as substitui pelo valor literal definido, de forma cega.
 - **Sintaxe:** `#define ledPin 13` (Atenção: não utiliza sinal de igual `=` e não termina com ponto e vírgula `;`).
 - **Vantagem:** **Não consome nenhum byte da memória SRAM**, pois seus valores são incorporados diretamente como constantes literais nas instruções binárias da Flash.

- **O Qualificador `const`:** Declara uma variável convencional de leitura e escrita padrão, porém qualifica seu estado de uso como **somente leitura**. Seu valor deve ser obrigatoriamente atribuído no exato momento da declaração e não poderá ser alterado posteriormente pelo programa.
 - *Sintaxe:* `const byte ledPin = 13;`
 - *Vantagem:* Respeita regras de escopo e tipagem estrita de variáveis em C, facilitando a depuração segura de código pelo compilador.
-

Bloco 3 — Operadores Matemáticos, Lógicos, de Atribuição e Precedência

3.1 Operadores Aritméticos e Divisões

Os operadores aritméticos realizam cálculos matemáticos sobre valores numéricos de variáveis e constantes:

- `+` (Adição)
- `-` (Subtração)
- `*` (Multiplicação)
- `/` (Divisão)
- `%` (Módulo ou Resto da Divisão) — **Aplicável unicamente para operandos do tipo inteiro** (`int` ou `char`).

A Armadilha da Divisão Inteira:

Em C, se os dois números operados forem inteiros, o compilador realizará uma divisão inteira por padrão, **descartando completamente a parte fracionária do quociente** (sem realizar arredondamentos).

```
int resultado1 = 5 / 2; // O resultado gravado na variável será 2.
```

```
float resultado2 = 5 / 2; // O resultado continuará sendo 2.0 (pois a divisão entre inteiros ocorreu antes)!
```

```
float resultado3 = 5.0 / 2.0; // O resultado correto será 2.5 (pois ao menos um dos valores foi forçado como float).
```

3.2 Operadores Relacionais e Lógicos

Os operadores relacionais e lógicos comparam valores ou expressões e retornam obrigatoriamente um valor booleano: **1 (verdadeiro)** ou **0 (falso)**.

Operadores Relacionais (Comparação):

- `==` (Igual a) — *Cuidado para não confundir com o operador de atribuição de sinal único `=`.*
- `!=` (Diferente de).
- `>` (Maior que) / `<` (Menor que).
- `>=` (Maior ou igual a) / `<=` (Menor ou igual a).

Operadores Lógicos (Conexão de Condições):

Permitem unificar múltiplas comparações relacionais em expressões mais complexas:

- **`&&` (Operador E / AND):** A expressão composta resultante só retornará Verdadeiro (1) se **todas as condições individuais** que a compõem forem verdadeiras.
- **`||` (Operador OU / OR):** A expressão retornará Verdadeiro (1) se **pelo menos uma das condições** individuais for verdadeira.

- **! (Operador NEGAÇÃO / NOT):** Inverte o valor lógico da expressão subsequente.
-

3.3 Incremento, Decremento e Atribuições Compactas

Para otimizar a escrita de códigos repetitivos, a linguagem C provê formas sintéticas e eficientes de operação e atribuição imediata:

Operadores de Atribuição Composta:

Substituem a repetição da variável de destino na expressão de cálculo:

- $x += y$ equivale a $x = x + y$
- $x -= y$ equivale a $x = x - y$
- $x *= y$ equivale a $x = x * y$
- $x /= y$ equivale a $x = x / y$
- $x \% = y$ equivale a $x = x \% y$

⚠ Atenção para a Precedência Oculta: O operador de atribuição composta atua sobre o **resultado total** da expressão localizada à sua direita. Portanto: $x *= y - 10$; é o equivalente exato a $x = x * (y - 10)$; (e **não** a $x = x * y - 10$;;).

Operadores de Pré e Pós-Incremento/Decremento:

Utilizados para somar ou subtrair uma unidade (1) de uma variável de forma rápida:

- **Pós-Incremento ($x++$):** Utiliza o valor atual de x na expressão e, em seguida, acrescenta 1 à variável.
- **Pré-Incremento ($++x$):** Adiciona 1 à variável x imediatamente e depois utiliza o valor já incrementado para computar a expressão.

```
int x = 10;
```

```
int y = x++; // y recebe 10 (valor original) e x vai para 11.
```

```
int a = 10;
```

```
int b = ++a; // a vai para 11 e b recebe 11 (valor já incrementado).
```

3.4 Tabela de Precedência de Operadores

Quando uma mesma linha de comando possui múltiplos operadores, o compilador decide qual cálculo executar primeiro com base em uma hierarquia de prioridades fixa (regras de precedência). O uso de **parênteses ()** permite forçar e controlar a ordem em que os cálculos ocorrem.

Abaixo estão listados os níveis mais comuns de precedência na linguagem C (do de maior prioridade para o de menor):

1. $++$ (Pré-incremento) e $--$ (Pré-decremento), Parênteses (), Elemento de array [].
2. $++$ (Pós-incremento) e $--$ (Pós-decremento), Operador de negação lógica !.
3. $*$ (Multiplicação), $/$ (Divisão), $\%$ (Módulo ou Resto).
4. $+$ (Adição), $-$ (Subtração).
5. $<<$, $>>$ (Deslocadores de bits).
6. $<$, $<=$, $>$, $>=$ (Operadores relacionais de magnitude).

7. `==`, `!=` (Operadores relacionais de igualdade/diferença).
 8. `&&` (Operador lógico E).
 9. `||` (Operador lógico OU).
 10. `=`, `+=`, `-=`, `*=`, `/=`, `%=` (Operadores de atribuição simples e compostas).
-

Resumo das Boas Práticas da Aula

1. **Sempre prefira o tipo de dado de menor tamanho possível** para economizar a preciosa memória SRAM do microcontrolador (ex: use `byte` para pinos).
 2. **Declare variáveis com escopo local** sempre que possível para que a memória SRAM seja liberada ao fim das funções.
 3. **Utilize `#define`** para associar nomes fáceis a pinos físicos do circuito sem desperdiçar memória RAM de variáveis.
 4. **Cuidado redobrado com a divisão inteira** e garanta que ao menos um elemento seja representado com ponto decimal (como `.0`) quando necessitar de resultados com casas fracionárias.
-

Bloco 4 — Laboratório Prático e Revisão de Código Limpo

Esta atividade foi projetada para consolidar a compreensão sobre o gerenciamento rigoroso da memória SRAM, a escolha adequada de tipos primitivos, o escopo de variáveis e a aplicação de conversão de tipos (casting) em sistemas de recursos restritos.

4.1 Análise de Código Legado e Refatoração Guiada

Objetivo Pedagógico: Desenvolver a capacidade crítica de identificação de más práticas de programação embarcada, compreendendo os impactos diretos do uso inadequado de memória no comportamento físico do microcontrolador (ATmega328P).

4.1.1 Código Legado Ineficiente

Examine o código fonte abaixo, extraído de um projeto legado de monitoramento analógico:

```
int pinoLeitura = A0; // Pino do sensor analógico
int pinoLed = 13;    // Pino do LED de alerta
int valorLido = 0;   // Leitura bruta do ADC
float tensao = 0;    // Tensão calculada em Volts

void setup() {
    Serial.begin(9600);
    pinMode(pinoLed, OUTPUT);
}

void loop() {
    valorLido = analogRead(pinoLeitura);
    tensao = valorLido / 1023 * 5; // Cálculo da tensão equivalente
```



```
if(tensao > 2.5) {  
    digitalWrite(pinoLed, HIGH);  
} else {  
    digitalWrite(pinoLed, LOW);  
}  
delay(1000);  
}
```

4.1.2 Atividade: Análise e Refatoração Guiada de Código Legado

Vocês devem analisar o código mal estruturado acima, identificar cinco falhas de boas práticas/otimização e reescrevê-lo aplicando os princípios de código limpo embarcado.

4.2 - Desafio Prático: Refatoração de Sistema Térmico e Monitoramento Temporizado

Objetivo Pedagógico: Desenvolver a capacidade de identificar falhas críticas de gerenciamento de memória SRAM, erros de divisão inteira, estouro de variáveis de tempo e violações de escopo em sistemas com microcontroladores de recursos restritos (como o ATmega328P).

Esta atividade foca em **diagnóstico técnico, análise de consumo de memória SRAM, correção de bugs de divisão inteira/estouro de tempo e refatoração com código limpo**.

4.2.1 Contexto do Problema e Código Legado Ineficiente

Um desenvolvedor iniciante projetou um módulo de monitoramento de sobreaquecimento para um painel industrial alimentado por um microcontrolador ATmega328P. O sistema lê um sensor analógico de temperatura no pino **A1**, aciona um ventilador (*cooler*) no pino digital **8**, pisca um alarme sonoro/visual no pino **9** quando a temperatura ultrapassa 45 °C e contabiliza a quantidade de acionamentos do alarme.

O código funciona de forma errática: a leitura de temperatura resulta constantemente em **0 °C**, o temporizador do alarme trava após alguns segundos de funcionamento e o consumo de memória RAM do sistema está desnecessariamente elevado.

Código Legado:

// --- Código Legado Ineficiente ---

```
int PINO_SENSOR = A1;    // Pino do sensor de temperatura  
int PINO_COOLER = 8;     // Pino do cooler  
int PINO_ALARME = 9;     // Pino do alarme
```

```
int leituraBruta = 0;     // Leitura do ADC  
float temperatura = 0;    // Temperatura calculada em Celsius  
int contadorAlertas = 0;  // Contagem de alertas  
int tempoAnterior = 0;    // Armazena tempo em ms para o alarme
```

```
void setup() {  
    Serial.begin(9600);
```



```
pinMode(PINO_COOLER, OUTPUT);
pinMode(PINO_ALARME, OUTPUT);
}

void loop() {
    leituraBruta = analogRead(PINO_SENSOR);

    // Conversão da leitura do ADC (0 a 1023) para graus Celsius (0 a 100 °C)
    temperatura = (leituraBruta / 1023) * 100;

    if (temperatura > 45) {
        digitalWrite(PINO_COOLER, HIGH);

        // Incrementa contagem de alertas
        contadorAlertas = contadorAlertas + 1;

        // Pisca o alarme a cada 500 ms usando millis()
        if (millis() - tempoAnterior > 500) {
            digitalWrite(PINO_ALARME, HIGH);
            tempoAnterior = millis();
        }
    } else {
        digitalWrite(PINO_COOLER, LOW);
        digitalWrite(PINO_ALARME, LOW);
    }
}
```

4.2.2 Atividade

Com base no conteúdo teórico da Aula 04, realize as três etapas a seguir:

1. **Análise Crítica e Diagnóstico:** Identifique e explique pelo menos **5 falhas de programação** presentes no código legado (considerando tipos de dados, escopo de variáveis, matemática em C, estouro de variáveis e boas práticas de código limpo embarcado).
2. **Cálculo de Consumo de Memória SRAM:** Calcule exatamente quantos *bytes* de SRAM são alocados pelas variáveis globais no código legado e determine quantos *bytes* de SRAM são salvos após a refatoração.
3. **Refatoração do Código Fonte:** Reescreva o *sketch* completo aplicando:
 - Substituição de pinos fixos por diretivas `#define` ou qualificadores `const`.
 - Escolha dos menores tipos primitivos adequados (`byte`, `unsigned long`, etc.).
 - Ajuste de escopo (uso de variáveis locais e locais estáticas `static`).
 - Correção do cálculo aritmético para evitar a armadilha da divisão inteira.
 - Utilização de operadores compactos de incremento/atribuição.
 - Convenção de nomenclatura *CamelCase* e comentários legíveis